PY102: ALGORITHMIC COMPLEXITY & DATA STRUCTURES

WEEK 3 MASTER COMPENDIUM (LESSONS 1-9)

EXECUTIVE COURSE LEVEL COMPENDIUM

This compendium compiles all essential lecture notes, conceptual breakdowns, code implementations, and algorithmic design trade-offs from **Week 3 (Lessons 1-9)** of the PY102 syllabus. It is structured as a high-fidelity reference manual suitable for immediate integration into your master software engineering notebook. Each lesson is presented comprehensively with exhaustive code examples and complexity analysis to ensure complete informational preservation.

PART I: FOUNDATIONS OF ALGORITHMIC COMPLEXITY

1.1 The Three Pillars of Code Quality (Lesson 1)

A successful algorithm must be evaluated against three core software engineering criteria. While they are sometimes in tension, they must be balanced in professional code:

• **Correctness:** Always giving the correct answer for every valid input. *No exceptions.* If an algorithm is fast and beautiful but sometimes wrong, it is dangerous or useless. Correctness is the absolute prerequisite.
• **Efficiency:** The volume of work (measured in basic computational execution steps) an algorithm performs. Fewer execution steps translate directly into better system performance, lower CPU workloads, and reduced wait times.
• **Readability:** The ease with which human engineers can read, comprehend, and modify the source code. High readability ensures long-term maintainability, whereas correct but unreadable 'clever' code is a liability.

Demonstration: The Step-Counting Metaphor

To ground efficiency, consider a function that computes the sum of elements in a list. We analyze the algorithm's performance by counting basic execution steps as a function of the input size:

```
def sum_list(nums):
    total = 0 # 1 step (set total variable)
    for n in nums: # loop setup
        total += n # 1 step * n times (each loop iteration)
    return total # 1 step (return statement)
```

For an input list of length **n = 3** (e.g., [5, 8, 2]), the step count is: 1 (initialization) + 3 (loop additions) + 1 (return) ≈ 5 steps. If **n = 6**, the loop runs more times and the step count rises to 1 + 6 + 1 = 8 steps. Thus, as input size grows, the number of loop steps grows linearly with it. Two correct algorithms can solve the same problem but perform vastly different amounts of work. For example, a wasteful nested loop could take **n × n** steps to do the same task.

Readability in Action: Self-Documenting Code

Consider two functions that compute identical logic. The messy version uses cryptic variable names, while the clear version tells a story and is significantly easier to maintain six months down the line:

```
# Messy and Cryptic Version
def f(a):
    s = 0
    for x in a: s += x
    return s
```

```
# Clear and Self-Documenting Version
def sum_list(nums):
    total = 0
    for num in nums: total += num
    return total
```

1.2 Complexity and User Experience (Lesson 2)

Algorithmic complexity defines how quickly the work (computational steps or execution time) grows when an algorithm is given more data. It determines whether an application feels instantaneous or painfully slow to the user.

• **Gentle Growth (Linear):** Doubling the input data size roughly doubles the steps. The application remains fast and responsive.
• **Explosive Growth (Quadratic/Exponential):** Doubling the input data size causes steps to multiply by 4x, 8x, or 16x. This rapid escalation causes CPU workload spikes and app lagginess.

> **Real-World Metaphor: Game Performance**
> Imagine a video game loading monsters: with 20 monsters, loading takes 1 second. With 40 monsters, loading takes 4 seconds. The input data (monsters) doubled (2x), but the work/time quadrupled (4x). This suggests the algorithm's complexity grows as a square of the input size, directly impacting the player's user experience.

1.3 Introduction to Big O Notation (Lesson 3)

Big O Notation is a standardized mathematical framework used to describe how the step count of an algorithm grows as the input size (denoted as **n**) increases. It focuses on the general shape of growth rather than exact, hardware-dependent seconds.

• **Constant Time - O(1):** The work does not change when the input grows. For example, accessing an element in a list directly by its index or printing a fixed number of items.

```
def show_first_item(numbers):
    print(numbers[0]) # Always exactly 1 access + 1 print step
def get_first_two(numbers):
    print(numbers[0])
    print(numbers[1]) # Always exactly 2 prints. Fixed work, so O(1).
```

• **Linear Time - O(n):** Steps grow in direct proportion to input size *n*. Typically characterized by a single loop going through the list once. Double the size means double the work.

```
def print_all(numbers):
    for n in numbers: print(n) # Runs exactly 'n' times (once per item)
```

• **Quadratic Time - O(n²):** Steps grow like a square ($n \times n$). Typically characterized by nested loops over the same data set, where for each item we scan the entire list again.

```
def print_pairs(numbers):
    for a in numbers:
        for b in numbers:
            print(a, b) # If n=3, prints 3x3=9 times. If n=4, prints 4x4=16 times.
```

1.4 Mathematical Rules of Big O (Lesson 4)

When performing professional complexity analysis, we must apply two fundamental mathematical rules to determine the overall Big O classification of an algorithm:

1. Ignore Constant Multipliers: Multipliers are ignored because they represent constant differences. For example, 2 * n, 5 * n, and n are all categorized in the **O(n)** linear growth class. The core principle is: if *n* doubles, steps roughly double.

2. Focus on the Dominant Term: We ignore small extra chunks (lower-order terms) and focus on what grows fastest as *n* becomes huge. For instance, n + 2, n + 100, and 5n + 7 all simplify to **O(n)**. A complexity expression of $n^2 + 2n + 100$ simplifies to **$O(n^2)$** because the n^2 term dominates completely at scale.

1.5 Analysis Scenarios: Best-Case vs. Worst-Case

Algorithms often behave differently depending on the exact input structure. We analyze two boundaries:

```
def contains(nums, target):
    for x in nums:
        if x == target: return True # Stop immediately upon finding target
    return False
```

• **Best-Case Scenario - O(1):** The target is the first element in the list. We check 1 item and stop. This is constant work, regardless of list size.
• **Worst-Case Scenario - O(n):** The target is missing or at the very end of the list. We must scan all *n* items. Because software must be robust under load, **Big O notation defaults to worst-case analysis unless specified otherwise.** Therefore, contains is officially classified as **O(n)**.

PART II: PERFORMANCE OF CORE DATA STRUCTURES

2.1 Python Lists: Position-Based Collections (Lesson 7)

A Python list is an ordered, position-based collection. It is designed for use cases where the order of elements or access by exact index position is critical.

• **Fast Operations:** Indexing (items[i]) takes constant **O(1)** time because the computer can jump straight to that slot in memory, regardless of list size. Appending at the end (items.append(x)) is also highly efficient, taking amortized **O(1)** time.
• **Slow Operations:** Inserting (items.insert(i, x)) or Removing (items.pop(i)) in the middle is **O(n)** because the computer must shift all subsequent elements to make room or close the gap . Searching for a value by scanning the list is also **O(n)**.

2.2 Python Dictionaries: Key-Value Mappings

A Python dictionary is an unordered collection that maps unique keys to values. It is engineered for extremely fast data retrieval when you identify items by unique labels (names/keys) rather than positions.

• **O(1) Performance:** Key lookups (d[key]), inserting/updating by key (d[key] = value), and testing membership (key in d) all execute in **O(1) average time**. Unlike lists, dictionaries do not suffer from index-shifting overhead.

2.3 Operations Complexity Comparison Matrix

Data Structure	Operation	Example Python Code	Time Complexity
List	Index by position	x = items[i]	O(1)
List	Append at end	items.append(x)	O(1) (amortized)
List	Insert/Remove (middle)	items.insert(i, x) / pop(i)	O(n) (shifting bottleneck)
List	Search by value	for x in items: compare	O(n) (linear scan)
Dict	Lookup by key	x = d[key]	O(1) (average)
Dict	Insert or Update by key	d[key] = value	O(1) (average)
Dict	Membership test	key in d	O(1) (average)

2.4 Stacks: Last-In, First-Out (LIFO) Operations (Lesson 8)

A stack implements Last-In, First-Out logic. The classic metaphor is a stack of cafeteria plates [98]. You add plates and remove plates strictly from the top (one end only).

In Python, a standard list is perfect for modeling a stack. We use .append(x) (push) to add an item to the end, and .pop() (pop) with no index to remove the last item. Because both actions occur at the very end of the list, no shifting of elements is required, yielding highly optimized **O(1)** time for both operations .

2.5 Queues: First-In, First-Out (FIFO) Operations

A queue implements First-In, First-Out logic, modeled as a line of people waiting at a ticket booth [98]. Elements are added to the back (enqueue) and removed from the front (dequeue).

• **The List Queue Anti-Pattern:** Implementing a queue with a normal list using .append(x) (back, O(1)) and .pop(0) (front, O(n)) is a massive performance bottleneck. Calling pop(0) forces the computer to slide every single remaining element left to close the gap, making it an expensive **O(n)** operation .

• **The collections.deque Solution:** Python provides a double-ended queue class (**collections.deque**) specially optimized for both ends. Adding to the back (append()) and popping from the front (popleft()) both run in true constant **O(1)** time without any element shifting .

2.6 Sequential Structures Complexity Matrix

Sequential Pattern	Core Logic	Optimal Structure	Operations	Complexity
Stack	LIFO (Cafeteria plates)	Python List	append(x) / pop()	O(1) / O(1)
Queue (Inefficient)	FIFO (Ticket line)	Python List	append(x) / pop(0)	O(1) / O(n) (shifting bottleneck)
Queue (Optimal)	FIFO (Ticket line)	collections.deque	append(x) / popleft()	O(1) / O(1) (no shifting)

PART III: ALGORITHMIC OPTIMIZATION & REFACTORING

3.1 Engineering Trade-Offs & Scaling (Lesson 5)

A critical engineering skill is deciding when to prioritize raw computational speed (time complexity) versus code simplicity and human readability.

• **Small, Rare Inputs (e.g., 5 to 30 elements):** Code readability and simplicity should win. In a script executed occasionally or read by interns, a slightly slower but more readable solution is preferred. The execution time difference is a fraction of a millisecond, making performance optimizations invisible.
• **Large, High-Frequency Inputs (e.g., 1,000,000-row log files):** Time complexity must win. An $O(n^2)$ algorithm on 1,000,000 entries requires one trillion checks, causing serious CPU bottlenecks and application freeze. An $O(n)$ algorithm completes the task in just one million checks. Efficiency is mandatory here.

3.2 Multi-Step Algorithms & Overall Complexity (Lesson 6)

Many algorithms involve multiple independent steps performed in sequence. We determine the overall Big O class by adding the complexities and keeping only the dominant term:

```
def print_summary(temps):
    print(temps[0]) # Step 1: O(1) index access
    for t in temps:
        print(t) # Step 2: O(n) single loop
                         # Overall: O(1) + O(n) -> Simplifies to O(n)
```

3.3 The Role of Early Exits in Performance

Implementing early exits (via the break statement) allows an algorithm to halt as soon as a condition is satisfied. This dramatically optimizes the **best-case** and average runtime but **leaves the worst-case complexity class unchanged** because in the worst-case scenario (e.g., the target is missing), the loop must still check every element to completion.

3.4 Key Refactoring Patterns (Lesson 9)

Refactoring is the process of restructured code to make it more efficient while preserving identical behavior. Below are three key refactoring patterns practiced in laboratory exercises:

Pattern A: Invariant Extraction (Loop Caching)

If a calculation's output does not change inside a loop, do not recompute it on each iteration. Compute it once before the loop, store the result in a variable (cache it), and reuse it inside the loop:

```
# INEFFICIENT: sum(numbers) is re-evaluated 'n' times inside loop
total = 0
for n in numbers:
    total += sum(numbers) # Repeated O(n) work inside the loop!
# OPTIMIZED: sum(numbers) is run exactly once
cached_sum = sum(numbers) # O(n) work done once
total = 0
for n in numbers:
    total += cached_sum # O(1) reuse inside the loop
```

Pattern B: Hash Lookups (Nested Loops to Dict)

Scanning a list inside a loop creates a nested $O(n^2)$ loop. By building a lookup dictionary once (O(n) time), we can perform O(1) average key lookups inside the loop, achieving a single-pass O(n) overall run-time:

```
# INEFFICIENT: O(n²) nested scanning
players = ['Ana', 'Ben', 'Cara', 'Dan']
scores = [10, 15, 8, 12]
requested = ['Cara', 'Ana', 'Dan']
for name in requested:
    for i in range(len(players)): # Inner loop scans players list
        if players[i] == name: print(name, 'has score', scores[i])
# OPTIMIZED: O(n) single pass with Dict lookup
score_dict = {players[i]: scores[i] for i in range(len(players))} # Build dict once
for name in requested:
    print(name, 'has score', score_dict[name]) # O(1) average lookup
```

Pattern C: Single-Pass Refactoring

When performing operations over linear sequences, eliminate redundant loops and nested iterations that execute duplicate runs over the same elements:

```
# INEFFICIENT: nested loop with useless inner loop breaking immediately
count = 0
for i in range(len(nums)):
    for j in range(len(nums)):
        if nums[i] > 10:
            count += 1
            break
# OPTIMIZED: clean single loop
count = 0
for i in range(len(nums)):
    if nums[i] > 10: count += 1
```

3.5 Case Study: Refactoring Chat App User Tracking

A highly practical optimization scenario involves tracking active online users in a chat server. For each incoming log event (e.g., login or logout), the application must check if the user is already online and modify the collection of online users. Below is the list-to-dictionary optimization progression:

The Bottleneck List-Based Code:

This version relies on checking name in online_users which requires scanning the list sequentially in a nested loop (O(n) work). Furthermore, logging a user out relies on online_users.remove(user), which also scans and shifts items under the hood. The cumulative workload is **O(n²) complexity**.

```
online_users = ['alice', 'bob', 'carol', 'dave']
events = [('login', 'erin'), ('logout', 'bob'), ('login', 'frank'), ('logout', 'carol'), ('logout',
'jack')]
for action, user in events:
    is_online = False
    for name in online_users: # Nested scanning loop (O(n))
        if name == user:
            is_online = True
            break

    if action == 'login' and not is_online:
        online_users.append(user)
    elif action == 'logout' and is_online:
        online_users.remove(user) # List removal (O(n) scans and shifts)
```

The Optimized Dictionary-Based Code:

By mapping the active online users in a dictionary (key: username, value: True), we can perform average **O(1) direct lookups** (using user in online) and O(1) updates (using online.pop(user) or online[user] = True). This eliminates the nested loop, converting the program into a highly scalable **O(n) linear complexity solution**:

```
online_users = ['alice', 'bob', 'carol', 'dave']
events = [('login', 'erin'), ('logout', 'bob'), ('login', 'frank'), ('logout', 'carol'), ('logout',
'jack')]
# 1) Build dictionary from initial list in O(n) time
online = {name: True for name in online_users}
# 2) Process events with O(1) average lookups & updates
for action, user in events:
    if action == 'logout' and user in online:
        online.pop(user) # O(1) dictionary pop
    elif action == 'login':
        online[user] = True # O(1) dictionary insertion

# 3) Rebuild final list from dictionary keys in O(n) time
final_online = [name for name in online]
print('Online now:', final_online) # Output: ['alice', 'dave', 'erin', 'frank']
```

Summary of Refactoring Benefits

By replacing the nested list scan with dictionary hashing, we achieved identical functional behavior while eliminating the CPU processing bottleneck. In high-scale systems with millions of daily event logs, this single change improves execution speed by orders of magnitude and reduces cloud infrastructure costs.